



A 2D-CNN-LSTM-Based Deep Learning Model for Forex Price Prediction using Lag Features

Bilguun Narmandakh¹ · Yuming Zhang¹ · Zhen Li² · Paul Anderson¹

Received: 25 January 2025 / Accepted: 26 August 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Foreign exchange (Forex) plays a critical role in the global financial market, and accurate forecasting of Forex price movements is essential for investors to manage risk and enhance profitability. However, due to the influence of numerous internal and external factors, Forex time series are inherently volatile and noisy, posing significant challenges for reliable prediction. Existing deep learning approaches often rely on conventional historical financial data and technical indicators, which may not effectively capture the underlying trends in the data. To address this limitation, we propose a novel deep learning model that leverages lag features - differences computed from historical times series values - as input. These features inherently encode trend information and are shown to be statistically stationary, making them more suitable for learning in deep neural networks. The proposed model integrates a 2D Convolutional Neural Network (2D-CNN), a Long Short-term Memory (LSTM) layer, and a dense output layer. The 2D-CNN, equipped with an asymmetric kernel, enables the extraction of features along both time and feature dimensions with different weighting sensitivities. Subsequently, the LSTM layer captures long-term temporal dependencies, while the dense layer produces the final trend prediction. We evaluate our method on four major currency pairs - EUR/USD, CAD/USD, GBP/USD, and JPY/USD. Our method achieves up to 82.3% accuracy for one-day-ahead predictions and up to 64% accuracy for five-day-ahead predictions. Compared to traditional deep learning models such as MLP, 2D-CNN, LSTM, and CNN-LSTM, our approach consistently delivers superior accuracy and faster convergence. These results demonstrate the effectiveness of using lag features and the proposed architecture for robust Forex trend forecasting. Enhanced forecasting accuracy achieved through lag features and deep learning models can offer significant practical benefits, particularly in the areas of trading strategy optimization and risk management.

Keywords Deep learning · 2D-CNN · LSTM · Lag features · Exchange rate prediction · Forex market prediction

Bilguun Narmandakh and Yuming Zhang contributed equally to this work.

Extended author information available on the last page of the article

1 Introduction

It has never been an easy task to forecast the future of the financial market, which is influenced by a great number of known and unknown variables mixing technical factors and human Sentiments. Foreign exchange rates, the rates at which currencies can be exchanged for one another, are among the hardest to predict. As the largest and most liquid market in the world, the global Forex market in 2020 was valued at \$2.4 quadrillion. Predicting Forex is imperative in the determination of cross-border trade and capital flow dynamics. Movements in Forex influence the decision-making of individuals, businesses, and governments. Forex forecasting is a cornerstone of most international transactions and thus critical for evaluating the returns and risks attached to the international business and economic environment.

It has been argued that the movement of a financial market follows a random walk process and can't be precisely modeled and predicted solely based on the past behaviors (Fama, 1995; Malkiel, 1999). Although the argument could be correct to some extent, recent research has proved that financial market forecasting is possible and the promising result can be verified with the help of advanced computer-aided techniques, such as machine learning (Di Persio et al., 2016, 2017; Gudelek et al., 2017; Jiang, 2021). The success is attributed to the advancement of powerful computer hardware and the massive data openly available.

The traditional analysis approaches to a financial market include fundamental analysis and technical analysis. The former focuses on the external factors, such as inflation, interest rates, politics, policy, international relationships, and many other variables that could influence the market (Abarbanell & Bushee, 1997; Wafi et al., 2015). The latter relies on the historical trading data in the market and various technical indicators that are computed from them (Han et al., May 2021; Taylor & Allen, 1992). Those technical indicators serve as trading signals that guide the investors to buy or sell the financial assets and gain profits. Both of the two methods are human intelligence based and the prediction accuracy is usually not satisfactory due to the high volatility nature of the financial market. In fact, it is not rare that the trading signals from different technical indicators are controversial and the investors still have to make a decision based on their professional experience and even intuitions. The limitation of the traditional methods implies that the current set of variables could be incomplete for forecasting, or there could be more critical and deeper indicative information in the historical data to be exploited and learned.

Researchers have dedicated great efforts to addressing this challenging problem. On one hand, various mathematical models based on statistical and probabilistic theories have been introduced, such as Vector Autoregression (VAR), Bayesian Vector Autoregression (BVAR), and Autoregressive Integrated Moving Average Mode (ARIMA) (Devi et al., 2013; Ariyo et al., 2014; Vantuch & Zelinka, 2015). While these models are effective in capturing linear relationships, they are limited in their ability to model the non-linear dynamics commonly present in financial time series. On the other hand, numerous data-driven machine learning based approaches have been developed, including the Support Vector Machine (SVM), k Nearest Neighbors (kNN), Decision Trees, and Fuzzy Time-Series (Rosillo et al., 2014; Sedighi et al., 2019; Kaur et al., 2016). In the past decade, neural network approaches have emerged

as transformative tools across many applications, including financial market forecasting (Kong et al., 2025; Miller et al., 2024). Generally, a neural network aims to learn a mapping function that predicts a target variable from input data with minimal error across all samples. This mapping function can be highly nonlinear and complex. Finding such a function essentially involves solving an optimization problem. To achieve this, a neural network is initialized with a large number of parameters-referred to as *weights*-with the goal of modeling a high-dimensional function space capable of representing arbitrarily complex relationships. An optimization algorithm, typically through the *backpropagation* technique, iteratively adjusts these weights by processing training data, minimizing the prediction error. This process, known as *training*, results in a trained neural network that approximates the underlying mapping. The trained model can then be applied to predict unseen data, such as future movements in financial markets. Historically, the neural network approach faced challenges due to extensive computational requirements and the need for large volumes of training data. However, recent advancements in hardware-particularly Graphics Processing Units (GPUs)-and the proliferation of high-speed internet have significantly mitigated these obstacles. Consequently, a multitude of neural network models have been developed, demonstrating superior performance over traditional machine learning techniques in financial forecasting tasks.

Despite promising results, neural network models face two key challenges when applied to financial market forecasting. First, selecting appropriate input features is crucial, but often done haphazardly. Common features include historical trading data, technical indicators, and indices from other financial markets, along with their combinations (Alzubaidi et al., 2021). However, a systematic approach to feature selection is lacking, and the impact of specific features on forecasting accuracy remains largely unknown. Second, designing an effective neural network model itself presents a challenge (Fogel, 1991; Anders & Korn, 1999). Neural networks are inherently application-specific. For the same forecasting task with identical input data, different neural network architectures, such as Multi-Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), 2D-CNN, Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM) networks, and their various combinations, can exhibit markedly different performance, owing to their distinct pattern recognition capabilities. Designing an effective neural network involves numerous choices, including the type of layers used, the number of perceptrons per layer, the number of hidden layers (i.e., the network's depth), and the arrangement or cascading order of these layers. Currently, successful models are often developed empirically, relying on trial-and-error rather than rigorous mathematical principles. In this study, we aim to address these two critical challenges.

We propose a novel deep learning framework that leverages lag features for effective Forex prediction. Exchange rates are inherently non-stationary, which presents significant challenges for traditional forecasting models that rely on stationarity assumptions. Such models often perform poorly when applied directly to raw, noisy financial data (Assaf, 2008; Rodrigues et al., 2020). In contrast, lag-feature-based deep learning models offer a practical alternative for handling noisy, non-stationary financial data (Khatun et al., 2024). By applying differencing, these models help transform time series data into a more stationary form, an essential condition for

many predictive techniques to be effective. Lag features capture nonlinear dependencies within the data, allowing the model to reflect more intricate market dynamics. This approach avoids making strong assumptions about the underlying data distribution while still leveraging its temporal structure. Furthermore, the proposed method supports direct multi-horizon forecasting (e.g., predicting 1 to 5 days ahead), which helps reduce cumulative prediction errors across time steps (Moreno-Pino et al., 2023). Leveraging lag features improves the model's generalization capability and interpretability while also reducing the risk of overfitting.

In summary, our main contributions are as follows.

- Utilization of lag-k features:** We employ lag-k features as inputs to predict the Forex price trend over the next k days, where k is a positive integer. These features are essentially the k-th differences derived from the original historical trading data. By directly encoding the trend information between specific days, lag-k features are more accessible for machine learning models to learn. Statistical analysis using the Augmented Dickey-Fuller (ADF) test demonstrates that while the original data is non-stationary, the lag-k features are stationary, confirming their suitability for modeling.
- Development of a novel 2D-CNN-LSTM model:** We propose a deep learning architecture comprising a 2D Convolutional Neural Network (2D-CNN), an LSTM layer, and a dense output layer. Unlike traditional 1D-CNNs, the 2D-CNN captures patterns along both the time and feature axes, with an asymmetric kernel designed to accommodate different weighting ranges across these dimensions. The output from the 2D-CNN feeds into the LSTM, which further models temporal dependencies. The final dense layer with a sigmoid activation function produces the prediction. Our model demonstrates superior learning ability and has achieved promising results across several Forex forecasting tasks, including challenging multi-day predictions.
- Comprehensive experimental validation:** We conducted extensive experiments comparing our approach with classical deep learning models such as MLP, LSTM, 2D-CNN, and CNN-LSTM. All models were trained using both conventional technical indicators and lag-k features. Results consistently show that incorporating lag-k features significantly enhances prediction accuracy across all models. Furthermore, our proposed model outperforms the classical counterparts in both accuracy and convergence speed, confirming its effectiveness and efficiency.

The remainder of this paper is organized as follows. Section 2 reviews the relevant recent research developments in the field. Section 3 details the methodology adopted for developing the proposed framework, including an overview of fundamental neural network components and a comprehensive description of the architecture of our 2D-CNN-LSTM model. Section 4 presents the experimental setup, reports the results, and provides an in-depth analysis and discussion. Finally, Section 5 summarizes our work, discusses the potential of the proposed model, and outlines possible directions for future improvements. The paper concludes with Acknowledgments and References.

2 Related Work

Although Forex and stocks make up the world's two most popular markets, not as much attention has been paid to Forex prediction by computer scientists. Limited research has been done applying deep learning techniques to forecasting Forex even for the mostly traded currencies that include U.S. dollar, euro, Japanese yen, pound sterling, and the Canadian dollar. A survey on Forex and stock price prediction using deep learning showed this significant imbalance between the two strands of research (Hu et al., 2021). In particular, of the articles surveyed that used the CNN model and/or the LSTM model, almost all were on stock price prediction. This research aims at narrowing the gap and developing a novel deep learning model for effective Forex prediction.

The efforts for improving the financial market forecasting can be categorized into two classes. One focuses on developing advanced predictive models using a variety of mathematical tools. Neural networks have demonstrated great capability of problem solving and knowledge learning, which is important for Handling a highly complex non-linear systems like the financial market. In 1988, White (1988) pioneered the neural network techniques in the case of IBM common stock daily returns. In recent decades, the neural network approach has drawn more and more researchers' attention. The other class focuses on seeking effective features that facilitate the predictive model to improve forecasting. Features are usually obtained from the public available information sources. Some sources provide numerical values, such as the historical trading data and stock market indices. Although they can be fed directly to the predictive model, the technical indicators calculated from them are considered more indicative and easier for the model to process. Other sources include cooperation reports, financial news, posts and comments on social medias, etc. This type of information requires extra pre-processing to interpret and digitize the semantics (Khan et al., 2022; Zhao & Yang, 2023; Chandola et al., 2023). Effective input features should contain enough relevant information and be easy to recognize as well by the model. Such features can boost forecasting accuracy and reduce dimensionality, and hence save computational resources in both time and memory.

Di Persio et al. (2016) compared three classical neural networks, MLP, CNN, and RNN, on predicting the movement of S&P500 index. The features they used included PCA-transformed and tree-based-selected from the raw data. They also proposed a novel Wavelet+CNN model and the experimental results showed that the proposed model outperformed the three classical neural networks. Furthermore, they employed an ensemble of three weighted neural networks to improve prediction results. In their later work, Di Persio et al. (2017) compared three variant recurrent neural networks, basic RNN, LSTM, and GRU, on forecasting Google stock price movements within the next 15 days. They used the daily trading data, including the open, high, low, close price, and volume, in five years as the features. The best accuracy was achieved by LSTM at the 5-day prediction. Gudelek et al. (2017) proposed a 2D-CNN model to predict the stock movement trend. The 2D-CNN model requires 2D images input so as to learn locality along both the x- and y- axis. They clustered technical indicators to form square 2D images to feed the model. Their work outperformed the conventional Buy & Hold Strategy.

Hoseinzade and Haratizadeh (2019) employed a CNN-based framework to extract features from a set of 82 variables that included historical financial data, technical indicators, exchange rates of currencies, commodities, future contracts, and some big US company stock. The resulting features were a 104-component vector. Their method outperformed some baseline methods in F-measure (a variant of accuracy measure that is not affected by negatives). Nikou et al. (2019) compared four machine learning methods on prediction of the closing price of iShares MSCI United Kingdom. Their results showed that the recurrent network method with an LSTM block functioned better than other methods. Nabipour et al. (2020) compared 11 typical machine learning models, including a RNN and LSTM model, on stock market trend prediction. They used two classes of features, ten technical indicators calculated from the ten-year historical data of four stock market groups, and the binary values transformed from those technical indicators. The experimental results showed that the transformation was beneficial for improving prediction accuracy.

Kelotra and Pandey (2020) proposed the Rider-MBO optimization algorithm and used it in a Deep-ConvLSTM model for stock market prediction. They firstly calculated 12 technical indicators from the open and close prices. Then, the Sparse-FCM Clustering algorithm is applied to cluster the technical indicators into groups based on their similarities. The final features were selected from the centroid of each group. This process reduced the feature dimension and hence the computational complexity. Lu et al. (2020) developed a CNN-LSTM model that mainly consists of a 1D convolutional layer and a LSTM layer to predict the next-day closing price of Shanghai Composite Index. They used eight features, opening, closing, high, low, volume, turnover, up-and-down, and change, obtained from the historical data over 7172 trading days. Their model outperformed other classical models such as MLP, CNN, RNN, LSTM, and CNN-RNN. Lu et al. (2021) later proposed a CNN-BiLSTM-AM model, which presented performance improvement compared with the previous model.

Wang et al. (2021) proposed a novel CNN-TLSTM model for the USD/CNY exchange rate prediction. TLSTM is built by adding the 1-tanh function immediately after the input gate of the traditional LSTM. The modification was intended to change the range of the output of the input gate so as to retain the important features as much as possible. Their input dataset included the USD/CNY exchange rate, Nasdaq Index, Dow Jones Industrial Average, Shanghai Composite Index, and Hang Seng Index. The original data were further proceeded using the feasible region calculation formula to improve the stationary of the data. Their experiments showed that the proposed model outperform the conventional CNN-LSTM model. Wang et al. (2022) applied a novel transformer to predict main stock market indices worldwide, including CSI 300, S&P 500, Hang Seng Index, and Nikkei 225. The features were simply the close prices. Their experimental results showed that the transformer was superior to other classical deep learning models. Bhandari et al. (2022) utilized LSTM to predict the next-day price of the S&P 500 index using a combination of features selected from fundamental, macroeconomic, and technical data. In their study, LSTM was investigated in depth and various hyperparameters (optimizers, learning rates, batch sizes, etc) were carefully tuned and compared. They concluded that a single-layer LSTM achieved a higher accuracy than a multi-layer LSTM in the specific application.

Fathali et al. (2022) applied three models based on RNN, LSTM and CNN to predict future trends of NIFTY 50 stock prices. They selected different subset of the components in the historical data as the input features. The experimental results showed that LSTM fed with the feature set of High/Low/Open/Close (that is, excluding Volume) performed the best. Their study justified that redundant or irrelevant information could impair the training process and worsen the prediction accuracy. Sen et al. (2023) built five regression models based on CNN and LSTM to predict the open price of NIFTY 50 index, respectively. They mainly used the original historical data as the input features but also carried out a multistep forecasting with walk-forward validation for the CNN model. The results showed that the CNN model was the fastest and most accurate one. Ali et al. (2023) proposed a novel Akima-EMD-LSTM model to predict the KSE-100 close price of the Pakistan Stock Exchange. The raw data was decomposed using Empirical Mode Decomposition (EMD) with Akima spline interpolation, resulting in highly correlated sub-components that were then used to train the LSTM model. The empirical results demonstrated that the proposed approach outperformed all other conventional EMD-based models. Jarrah and Derbali (2023) utilized LSTM to predict the indices of the stock market of the Kingdom of Saudi Arabia (KSA) over the next seven days. The features came from the historical data from multiple representative stocks, namely, STC, Mobile, and Zain. The exponential smoothing technique was applied to remove the noise in the raw data. Zaheer et al. (2023) proposed a CNN-LSTM-RNN model that consists of three standalone models to predict the Shanghai Composite Index. After training, a selection method was applied to determine the best model according the R^2 metric on the validation. Their results showed that RNN performed the best. The features they used were simply the historical data.

In their most recent work, Billah et al. (2024) compared the simple SMA and EMA model with LSTM on predicting six stock prices using the historical data as the input features. Their results showed that the former was superior in long-term prediction, but LSTM performed better in short-term prediction. Fischer et al. (2024) applied the transformer architecture networks on a Forex market prediction task on the EUR/USD, USD/JPY, and GBP/USD currency pairs. They employed a long 1273-component feature vector that contains a variety of fundamental and technical indicators. The transformer model outperformed two benchmark models, a five-layer LSTM and a simple ARIMA model, by an evident margin. Navaei and Pahlevan-zadeh (2024) utilized three types of neural networks, RNN, LSTM, and GRU, to predict the future EUR/JPY price. Igboanusi (2024) implemented two LSTM models with 2 and 3 hidden layers to predict EUR/USD price. Both of them used historical data as the input features.

Our work is grounded in the hypothesis that lag features contain abundant trend information, making them highly beneficial for deep learning models to learn meaningful patterns for trend prediction. Unlike prior studies such as Livieris et al. (2020); Wang et al. (2021), which primarily utilized the first difference-or lag-1 feature-our approach extends this concept by incorporating up to lag-k features tailored to the specific prediction task. In this study, we employ a set of lag-1 through lag-5 features to forecast the next five days, and our framework can be readily adapted for weekly or monthly predictions as needed. Furthermore, we develop a novel 2D-CNN-LSTM

deep learning architecture. The 2D-CNN enables the extraction of features along both the time and feature axes, similar to the approach in Gudelek et al. (2017). However, for time series data, it is crucial to carefully exploit the temporal dependencies embedded in the sequential order of samples. In our model, the output of the 2D-CNN is further processed by an LSTM layer to explicitly capture these temporal patterns—a step that is absent in Gudelek et al. (2017). To the best of our knowledge, these contributions—namely, the extended use of lag-k features and the integrated 2D-CNN-LSTM architecture—distinguish our work from existing research in this area.

3 Methodology

In this section, we present the proposed framework, including details of the dataset, the lag features extraction process, and the model architecture. Additionally, a brief review of the necessary background knowledge, such as CNN and LSTM, is provided.

3.1 Data Collections

The dataset utilized in this study is based on historical EUR/USD exchange rates obtained from [Yahoo Finance](#). It includes daily open, high, low, close, adjusted close, and volume data. Since the adjusted close and volume are either redundant or consist entirely of zeros, they were excluded from the analysis. The dataset covers the recent decade, from May 13, 2013, to May 13, 2024, comprising 2,867 data points. The data were split, with the first 80% (up to March 2, 2022) used for training, and the remaining for testing. Figure 1 illustrates the historical data.



Fig. 1 The original historical data of EUR/USD exchange rates spans from May 13, 2013, to May 13, 2024. The split between the training and test sets occurs on March 2, 2022, as indicated by the vertical blue line

3.2 Feature Extraction

While numerical historical data is straightforward to use to train a learning model, it is often challenging for the model to extract indicative information and make accurate predictions. In practice, financial investors typically do not use raw historical data directly; instead, they compute more informative technical indicators from the data to serve as trading signals for their buy/sell/hold strategies. This approach is known as technical analysis. Inspired by this well-established practice, many researchers use technical indicators as input features to train their machine learning models (Gudelek et al., 2017; Hoseinzade & Haratizadeh, 2019; Kelotra & Pandey, 2020; Fischer et al., 2024). Commonly used technical indicators include DEMA, ROCr, ATR, RSI, SMA, ADX, CCI, MACD, OBV, SAR, EMA, T3, among others. Interested readers are referred to Kelotra and Pandey (2020) for detailed explanations of these indicators and their calculation formulas.

The reasons why historical data may not be suitable for directly fitting models can be complex, but one key factor is often its non-stationary nature. Briefly, a stochastic process generating a time series is considered stationary if its statistical properties remain constant over time (Livieris et al., 2020). Stationary processes are generally easier to analyze and model, making them more predictable. The stationarity of a time series can be assessed using the Augmented Dickey-Fuller (ADF) unit root test (Brockwell & Davis, 2002). We applied the ADF test to the historical EUR/USD exchange rates, and the results, shown in Table 1, indicate the presence of a unit root, meaning the series is non-stationary. Consequently, the raw historical data is not suitable for use as direct input features in model fitting.

To address the issue of non-stationarity, a common approach is the differencing transformation (Brockwell & Davis, 2002). Additionally, we observe that the differences between consecutive historical trading data inherently encode trend information, which can aid in trend prediction tasks. Based on this insight, we extract lag

Table 1 The ADF test results of the original historical data and lag-1 through lag-5 features

	Original historical data				Lag-1 features			
	Open	High	Low	Close	Open	High	Low	Close
ADF test								
t-stats	-1.765	-1.944	-1.789	-1.763	-23.274	-23.149	-22.326	-23.292
p-value	0.398	0.312	0.386	0.399	0.000	0.000	0.000	0.000
	Lag-2 features				Lag-3 features			
	Open	High	Low	Close	Open	High	Low	Close
ADF test								
t-stats	-9.635	-9.169	-8.891	-9.636	-9.147	-9.018	-8.951	-9.157
p-value	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	Lag-4 features				Lag-5 features			
	Open	High	Low	Close	Open	High	Low	Close
ADF test								
t-stats	-8.539	-8.489	-8.552	-8.528	-8.960	-9.271	-9.017	-8.955
p-value	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

features from the original historical data using the differencing transformation. The lag- k feature for the i -th date, denoted as $d_k^{(i)}$, is computed as:

$$d_k^{(i)} = v^{(i)} - v^{(i-k)} \quad (1)$$

where $v^{(i)} = [\text{open}^{(i)}, \text{high}^{(i)}, \text{low}^{(i)}, \text{close}^{(i)}]$ is a 1×4 row vector representing the trading data on the i -th date. Since our goal is to predict the short-term future close price up to five days ahead, we enhance the input features by including lag-1 through lag-5 features. The final feature vector for the i -th date, $x^{(i)}$, is constructed as:

$$x^{(i)} = [d_1^{(i)}, d_2^{(i)}, d_3^{(i)}, d_4^{(i)}, d_5^{(i)}] \quad (2)$$

This results in a feature vector with $4 \times 5 = 20$ components. The overall lag-1 through lag- k feature extraction process is summarized in Algorithm 1.

Algorithm 1 Lag 1- k Feature Extraction.

Require: $(v^{(1)}, v^{(2)}, \dots, v^{(n)}), k \geq 1$

Ensure: $(x^{(k+1)}, x^{(k+2)}, \dots, x^{(n)})$

```

1: for  $i = k + 1$  to  $n$  do
2:    $x^{(i)} \leftarrow$  an empty feature vector
3:   for  $j = 1$  to  $k$  do
4:      $x^{(i)}$  append  $v^{(i)} - v^{(i-j)}$ 
5:   end for
6: end for

```

We apply the ADF test to the lag features. The results clearly indicate that the data is stationary, as all p -values are effectively zero. Therefore, the lag features are suitable for use as input to a machine learning model. All ADF test results are summarized in Table 1, and the lag features are visualized in Fig. 2.

3.3 A Binary Classification Problem

In this work, we aim to predict the movement trend, either upward or downward, of the close price over the next five days, referred to as the day- l prediction, where $l = 1, 2, 3, 4, 5$. For example, when $l = 1$, the goal is to predict the trend of the following day's closing price. This constitutes a typical binary classification problem. The ground-truth label $y_l^{(i)}$, representing the trend for the i -th date, is calculated as:

$$y_l^{(i)} = \begin{cases} 1, & \text{close}^{(i)} \leq \text{close}^{(i+l)} \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Theoretically, the close price can be influenced by events occurring at any point in the past. However, in practical applications, we often adopt a simplified assumption: that only recent past events significantly impact the current price. In this study, we assume that the trend is affected by trading data from the past t days; for instance, we set $t = 30$. Accordingly, the input data sample corresponding to the label $y_l^{(i)}$ is

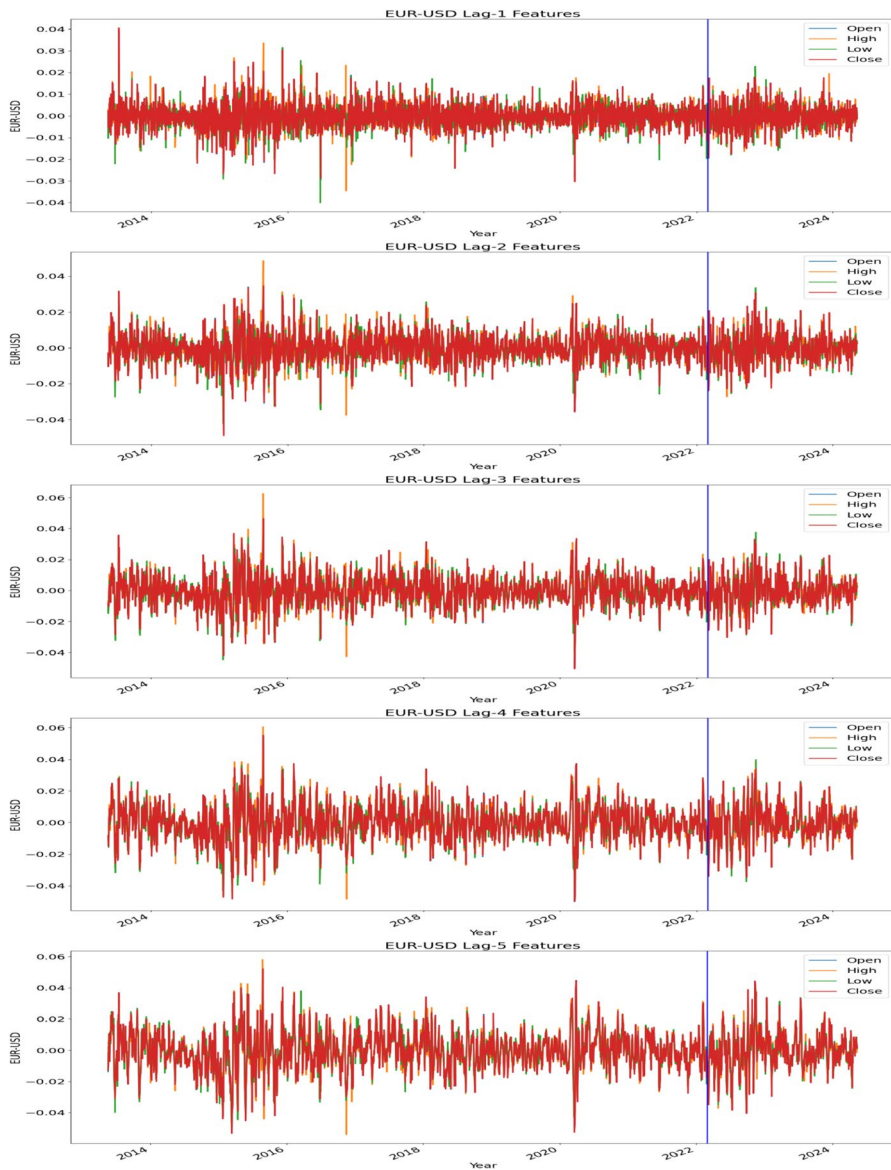


Fig. 2 The lag-1 through lag-5 features computed from the historical EUR/USD exchange rates

constructed as a feature matrix of size $t \times m$, where m is the number of components in each feature vector. This feature matrix is formulated as:

$$X^{(i)} = [x^{(i-t+1)}; x^{(i-t+2)}; \dots; x^{(i-1)}; x^{(i)}] \quad (4)$$

Our goal is to develop a logistic regression model that produces a prediction as follows:

$$\hat{y}_t^{(i)} = f(\Theta, X^{(i)}) \quad (5)$$

where $\hat{y}_t^{(i)}$ is the predicted value of the ground-truth trend $y_t^{(i)}$ defined in Eq. 3, and Θ is the vector of model parameters. The discrepancy between the predictions and the true labels is quantified using a loss function computed over the entire training set. A commonly used loss function for binary classification is the binary cross-entropy, given by:

$$J(\Theta) = \frac{1}{n} \sum_i [-y_t^{(i)} \log \hat{y}_t^{(i)} - (1 - y_t^{(i)}) \log(1 - \hat{y}_t^{(i)})] \quad (6)$$

The parameters Θ are optimized to minimize the loss function using algorithms such as SGD, RMSprop, Adam, and Adagrad.

3.4 CNNs

CNNs were introduced by LeCun et al. (1995) to address inherent limitations of classical MLPs. Unlike an MLP, which typically consists of multiple fully-connected (dense) layers, a CNN relies on convolutional layers. A convolutional layer computes a weighted sum over a local region of the input, called a *kernel*, using a sliding window approach. By moving the kernel across the entire input data, CNNs effectively capture topological information and local patterns that are invariant to position, resulting in the generation of a feature map. Each convolutional layer can contain multiple filters or channels, with each producing a separate feature map. Deep CNN architectures often stack multiple convolutional layers, enabling the model to learn highly complex and abstract features. Additionally, because kernels are small, the total number of trainable weights in a CNN is significantly lower than in an MLP, making CNN training more efficient and requiring less memory.

Due to its excellent pattern recognition capabilities, CNN has been applied to many challenging tasks, including time series forecasting. Currently, most researchers utilize 1D kernels, resulting in a 1D-CNN, applied to the feature vectors with the goal of capturing temporal patterns along the time axis. However, this approach inherently overlooks patterns that may exist across the feature axis. To address this limitation, we employ 2D kernels to capture patterns along both axes simultaneously. While a square kernel of size 3×3 is a common choice, as in Gudelek et al. (2017), our model adopts an asymmetric kernel designed to better capture patterns that may span a short range in one direction but extend longer in the other. Considering that the features could have correlations over a relatively larger range, we use an asymmetric kernel of size 3×5 in our model. Figure 3 illustrates the 1D, 2D symmetric, and 2D asymmetric kernels.

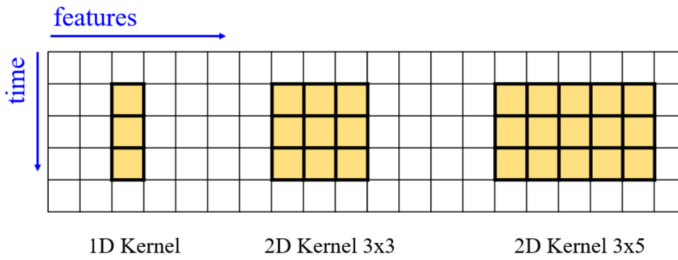


Fig. 3 A 1D kernel enables the model to learn patterns along a single axis, whereas a 2D kernel captures patterns along both axes simultaneously. Furthermore, an asymmetric 2D kernel allows the model to learn patterns that may extend over different ranges along each axis

3.5 RNNs and LSTM

RNNs are a specialized family of models well-suited for applications involving sequential data, such as time series. In such data, information is not only contained in the sample values but also in their order. RNNs maintain a hidden state that acts as a memory, capturing information from previous samples. The output at each time step is influenced by both the current input sample and the hidden state. Additionally, the hidden state is updated at each step based on the current sample, preparing it to incorporate information from subsequent inputs.

Although RNNs are effective at capturing the dynamics of sequential data, they suffer from the well-known vanishing and exploding gradient problems, especially when modeling long sequences. To address this issue, Hochreiter and Schmidhuber (1997) proposed the Long Short-Term Memory (LSTM) model in 1997. In addition to the hidden state used in classical RNNs, LSTM maintains an additional internal state and incorporates three multiplicative gates: the *input* gate, which controls whether the current sample influences the internal state; the *forget* gate, which determines whether to reset the internal state; and the *output* gate, which regulates how the hidden state is updated. This increased complexity enables LSTM to effectively mitigate the vanishing and exploding gradient problems. As a result, LSTM has demonstrated outstanding capability in learning from sequential data and is often the preferred choice when modeling sequences. In our study, the output from the 2D-CNN is treated as an initial feature representation, which is still in sequential form. This sequence is further processed by an LSTM layer to capture the temporal dependencies embedded in the order of the data.

3.6 The Proposed Model

As described in Sections 3.2 and 3.3, the input lag feature matrix $X^{(i)}$ is a 2D tensor of shape (30, 20), where 30 represents the number of past days considered, and 20 is the number of components in each feature vector. Consequently, the input to the model has shape (None, 30, 20), with None indicating the *batch size*. To utilize a 2D

convolutional layer, however, the input to the layer must be a 3D tensor. Therefore, we reshape the feature matrix to shape (None, 30, 20, 1), where the last dimension is designated as the *channel*. The reshaping layer is followed by the 2D Convolutional Neural Network that comprises four cascading 2D convolutional layers with 32, 64, 32, and 1 channel(s), respectively. All convolutional layers employ a kernel of size 3×5 and a tanh activation function. Notably, the final one-channel convolutional layer is immediately followed by another reshaping layer that reduces the output from a 3D tensor (30, 20, 1) to a 2D tensor (30, 20), which is necessary for the subsequent LSTM layer. The LSTM layer contains 64 units and is followed by a dense layer with a sigmoid activation function, producing the final trend prediction. To mitigate overfitting, techniques such as dropout and regularization are applied to all trainable layers. The total number of trainable parameters is 84,354. The detailed architecture of the proposed model is illustrated in Fig. 4.

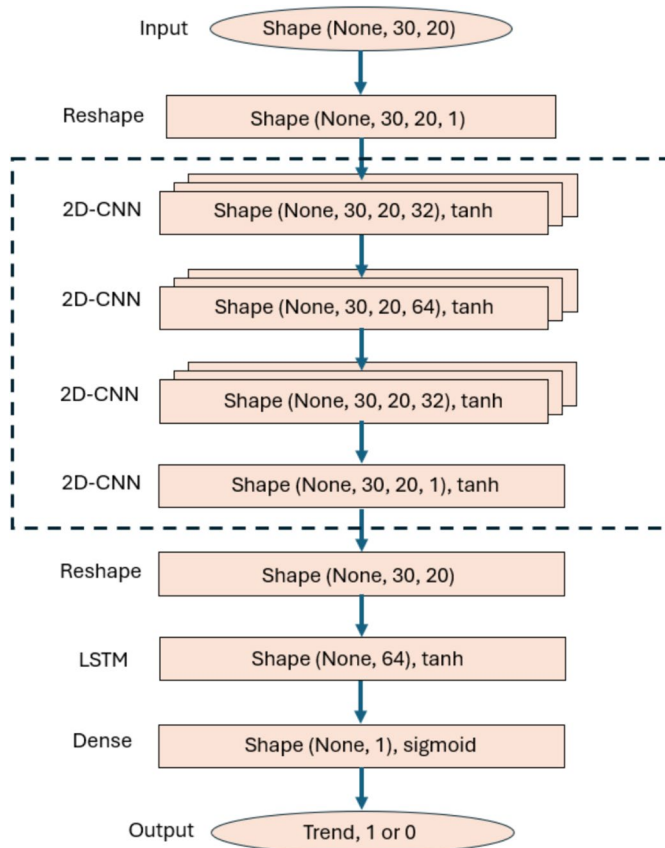


Fig. 4 The architecture of the proposed 2D-CNN-LSTM model. Note each input to the model is a 2D tensor of shape (30, 20), where 30 represents the number of past days considered, and 20 is the number of components in each feature vector

4 Experiments

We conduct extensive experiments to evaluate the effectiveness of the proposed methodology. Specifically, we compare four benchmark deep learning models, MLP, LSTM, 2D-CNN, and CNN-LSTM, with our proposed 2D-CNN-LSTM model. To validate the effectiveness of the lag features, all models are trained individually using two feature sets: the technical indicators and lag features, as described in Sections 3.1 and 3.2. The trained models are then employed to predict the movement trend of the EUR/USD closing price on the test set. Model performance is assessed based on the overall accuracy, precision, recall, and F1-score. Additionally, we apply our method to forecast three Forex currency pairs, CAD/USD, GBP/USD, and JPY/USD. As demonstrated by the experimental results, our approach proves to be effective, efficient, and reliable.

4.1 Experimental Setup

All experiments are conducted on a desktop computer with the hardware and software environment detailed in Table 2.

4.2 The Benchmark Models

We construct four benchmark deep learning models to compare with the proposed model, ensuring that all models have similar structures and depths for a fair comparison. Specifically, the MLP model comprises three dense layers with 32, 64, and 32 units, respectively. The LSTM model consists of three LSTM layers with the same number of units as the MLP layers. The CNN-LSTM model includes three 1D convolutional layers with 32, 64, and 32 units, respectively, followed by an LSTM layer with 64 units. The 2D-CNN model features four 2D convolutional layers identical to those in the 2D-CNN component of the proposed model. All models conclude with a dense layer containing a single unit with a sigmoid activation function to produce the trend prediction.

It is worth noting that all the aforementioned models are susceptible to overfitting, a common challenge in deep learning. Increasing the model depth can enhance its capacity to learn complex patterns; however, deeper models often achieve higher training accuracy while performing significantly worse on unseen test data. Our experiments suggest that using three hidden layers offers an appropriate trade-off

Table 2 The environment for all experiments

Hardware	Intel(R) Core(TM) i9-9900K CPU, Memory 64G, NVIDIA GeForce RTX 2080 SUPER
Operating System	Windows 11 Version 23H2
Software Tools	Tensorflow 2.10.0, Keras 2.10.0, Pandas 2.2.0, Sklearn 1.4.0, Statsmodels 0.14.1, Matplotlib 3.8.2, Talib 0.4.24

between model complexity and generalization in this study. Additionally, regularization techniques and dropout are employed in all models to mitigate overfitting.

4.3 Evaluation Metrics

As described in Section 3.3, predicting the movement trend, either upward or downward, of the close price is a typical binary classification problem. The performance of such models can be evaluated using accuracy, calculated as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \quad (7)$$

where TP (True Positive) is the number of instances correctly predicted to have an upward trend, TN (True Negative) is the number correctly predicted to have a downward trend, FP (False Positive) is the number of instances incorrectly predicted as upward when they are actually downward, and FN (False Negative) is the number incorrectly predicted as downward when they are actually upward. Accuracy is monitored during training and can serve as a signal for early stopping to prevent overfitting.

While accuracy is a suitable metric for balanced datasets, where the number of targets in each class is approximately equal, it may be misleading for unbalanced data. In such cases, metrics like precision, recall, and F1-score provide a more comprehensive evaluation. These are computed as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (8)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (9)$$

and the F1-score, which is the harmonic mean of precision and recall:

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

In this study, we evaluate all models using accuracy, as well as precision, recall, and F1-score, to ensure a comprehensive performance comparison.

4.4 Predicting the EUR/USD Price

Our task is to predict the movement trend of the EUR/USD closing price in the short-term future, up to five days ahead, using benchmark models and the proposed 2D-CNN-LSTM model. For each model, we use two sets of features as

input: popular technical indicators and lag features. The numerical values of these features often have different range of values, which can hinder model convergence or prevent the optimization process from finding optimal weights. This is a common problem in machine learning and necessitates feature scaling. To address this issue, we employ a common preprocessing scheme: normalizing the features into a uniform small interval, that is, (0, 1). The standard normalization formula is as follows:

$$X_{norm}^{(i)} = \frac{X^{(i)} - \mu}{\sigma} \quad (11)$$

where μ and σ are a tensor of the average value and the standard Deviation over the training features, respectively. We set the batch size to 256. When training the models, we use Adam as the optimizer with the learning rate of 0.0001.

4.4.1 Using Technical Indicators

Since raw historical data are non-stationary and contain limited informative value, they are not ideal as direct inputs to the models. Instead, technical indicators derived from the historical data are more suitable and widely used as features for model training (Gudelek et al., 2017; Hoseinzade & Haratizadeh, 2019; Kelotra & Pandey, 2020; Fischer et al., 2024). In this study, we employ a set of eleven common technical indicators, including DEMA, ROC, ATR, RSI, SMA, ADX, CCI, MACD, SAR, EMA, and T3, as the input features for the models. It is worth noting that the consequent input feature matrix becomes of size 30×11 for this case.

The training and test results using technical indicator features are presented in Fig. 5. Each row corresponds to one model predicting the trend for 1- to 5-day ahead periods, respectively. To facilitate clear and compact visualization, the loss and accuracy are plotted side by side, with identical axis scales across all plots for direct comparison. The number of training epochs varies among the models due to differences in their convergence speeds. In practice, early stopping can be employed to halt training when the loss and prediction accuracy no longer improve, thus avoiding unnecessary additional epochs and mitigating overfitting.

We observe that the LSTM model performs the worst among all, failing to produce promising results. In contrast, the MLP, 2D-CNN, and CNN-LSTM models achieve notable accuracy for 1-day and 2-day ahead predictions but do not perform as well for 3- to 5-day forecasts. The proposed 2D-CNN-LSTM model demonstrates promising results across 1-, 2-, and 3-day predictions, attaining the highest accuracy among all models. However, predicting 4- and 5-day trends remains challenging for all approaches, with none delivering satisfactory results. Notably, as we will discuss further, incorporating lag features can significantly enhance the performance of all models across all prediction tasks.

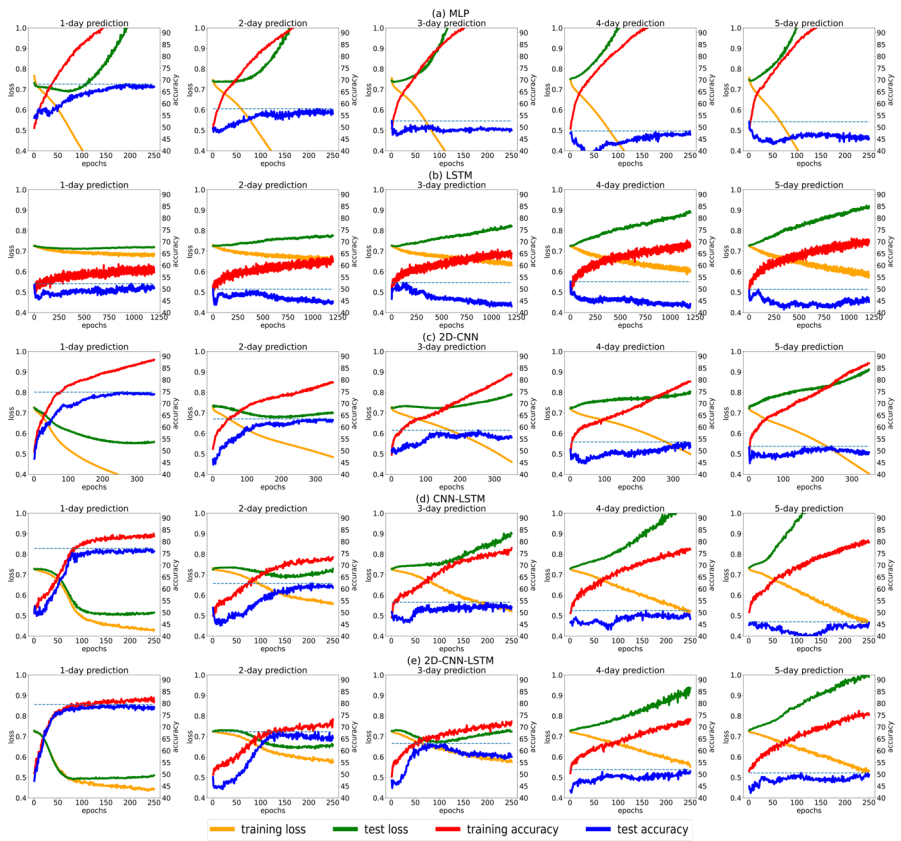


Fig. 5 Five models are employed to predict the EUR/USD price trend over the next five days using the eleven conventional technical indicators. From top to bottom, these models are: (a) MLP, (b) LSTM, (c) 2D-CNN, (d) CNN-LSTM, and (e) the proposed 2D-CNN-LSTM model. The dashed line indicates the highest accuracy achieved by each model on the test set

4.4.2 Using Lag Features

In this section, we maintain all existing settings but incorporate lag features into the training process, computed as described in Eq. 2 and normalized according to Eq. 11. The resulting training and test performance are shown in Fig. 6. It is evident that all models achieve significantly improved results compared to using only the conventional technical indicators. Specifically, although still delivering the worst performance, the LSTM model yields promising results for 1- and 2-day predictions. In contrast, the MLP performs well across 1- to 4-day forecasts, while the 2D-CNN, CNN-LSTM, and our proposed model deliver strong performance across all five prediction periods. Notably, our model outperforms all benchmark models with the highest accuracy and exhibits faster convergence. For instance, our model requires fewer than 25 epochs to converge for 1-day prediction (the first plot on the bottom of Fig. 6), which is substantially fewer than the epochs needed by other models.

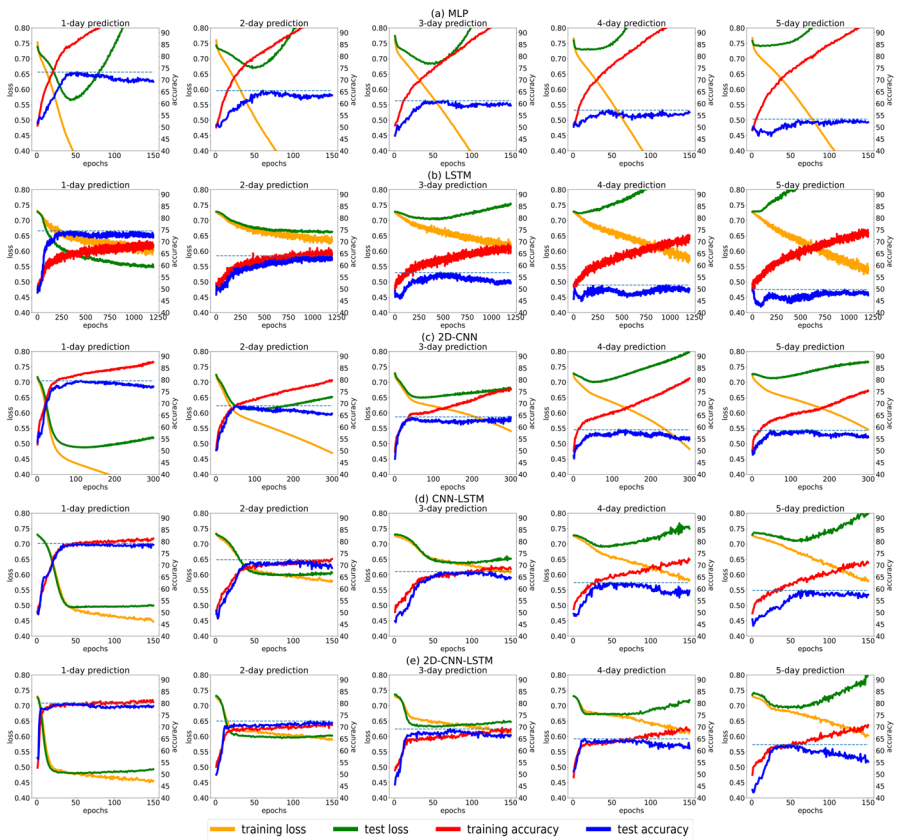


Fig. 6 Five models are employed to predict the EUR/USD price trend over the next five days using the lag features. From top to bottom, these models are: (a) MLP, (b) LSTM, (c) 2D-CNN, (d) CNN-LSTM, and (e) the proposed 2D-CNN-LSTM model. The dashed line indicates the highest accuracy achieved by each model on the test set

We select the model instance with the highest accuracy during training for each model and apply these instances to predict on the test set. Figure 7 displays the accuracy, precision, recall, and F1-score for each model instance, side-by-side. Our observations reveal that, for 1-day predictions, the 2D-CNN, CNN-LSTM, and 2D-CNN-LSTM models exhibit comparable performance across all four metrics. For 2-day predictions, the CNN-LSTM and 2D-CNN-LSTM models demonstrate superior performance over the others. Crucially, in the 3- to 5-day prediction, the proposed 2D-CNN-LSTM model consistently outperforms all other models by an evident margin. For instance, in the 5-day prediction, the proposed 2D-CNN-LSTM model achieves accuracy 0.625, precision 0.632, recall 0.579, and F1-score 0.605, while the regular CNN-LSTM model at the second place can only achieve 0.594, 0.602, 0.538, and 0.568, respectively. The comparison clearly highlights the capability of the proposed model in handling the more challenging short-term forecasting tasks.

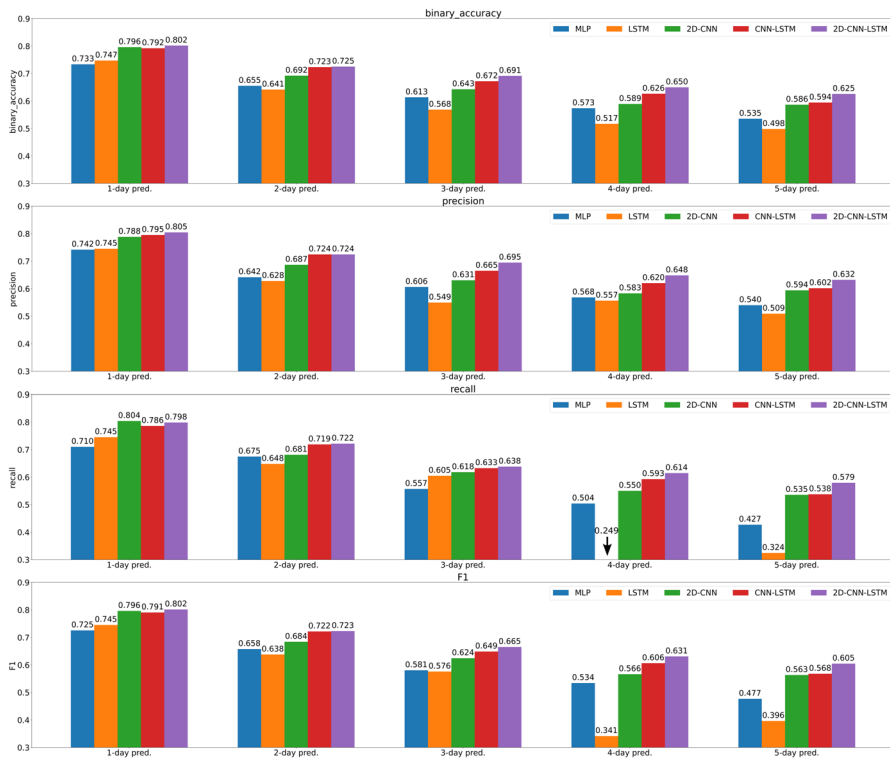


Fig. 7 Comparison of the accuracy, precision, recall, and F1-score of the five models using lag features

4.4.3 Summary

The highest prediction accuracies achieved by the models using the two different feature sets are summarized in Table 3. For comparison, we include the baseline accuracy obtained by a naive predictive model that always predicts the most frequent class. The performance of this naive model depends on the class distribution within the dataset. For instance, in the EUR/USD close price test set for 1-day movement trend prediction, approximately 50.5% of the samples are upward movements and 49.5% are downward. Consequently, the naive model can attain an accuracy of 50.5% simply by always predicting ‘up’. Interestingly, such a naive approach can sometimes be surprisingly difficult to outperform, especially when the class distribution is unbalanced. Nonetheless, an effective predictive model should achieve higher accuracy than this baseline to be considered useful. We can clearly observe that our proposed 2D-CNN-LSTM model is superior regardless of whether technical indicators or lag features are used as inputs. Importantly, using lag features as inputs significantly boosts the performance of all models. The 2D-CNN-LSTM model, especially when incorporating lagged features, achieves the best performance across all prediction tasks. For instance, even for the most challenging 5-day prediction, our approach attains an accuracy of 62.5%, surpassing the second-best method by a margin of 3%.

Table 3 The highest accuracy achieved by the models for the EUR/USD price trend forecasting using technical indicators (T.I.) and lag features, respectively

Models	features	1-day	2-day	3-day	4-day	5-day
Baseline	-	50.5	50.4	50.6	50.6	50.6
MLP	T.I.	68.2	57.8	52.6	48.6	52.3
	Lags	73.3	65.5	61.2	57.2	53.4
LSTM	T.I.	52.3	49.9	52.6	53.1	49.8
	Lags	74.6	64.0	56.9	51.7	49.7
2D-CNN	T.I.	74.8	63.5	58.7	53.7	51.9
	Lags	79.6	69.0	64.3	58.9	58.6
CNN-LSTM	T.I.	77.3	62.3	54.3	50.9	46.8
	Lags	79.2	72.3	67.3	62.6	59.4
2D-CNN-LSTM	T.I.	79.5	68.0	63.0	52.0	51.1
	Lags	80.1	72.5	69.1	65.0	62.5

4.5 Predicting three Additional Forex Prices

To further validate the robustness of our proposed method, we apply the 2D-CNN-LSTM model with lag features to predict three additional Forex pairs: CAD/USD, GBP/USD, and JPY/USD. The training and testing results are presented in Fig. 8, and the highest prediction accuracies are summarized in Table 4. Consistent with the EUR/USD results, the predictions are all promising. Notably, the highest accuracy reaches 82.3% for 1-day prediction of JPY/USD. For the more challenging 5-day prediction, our model achieves up to 64% accuracy for GBP/USD, demonstrating its effectiveness across different currency pairs and prediction horizons.

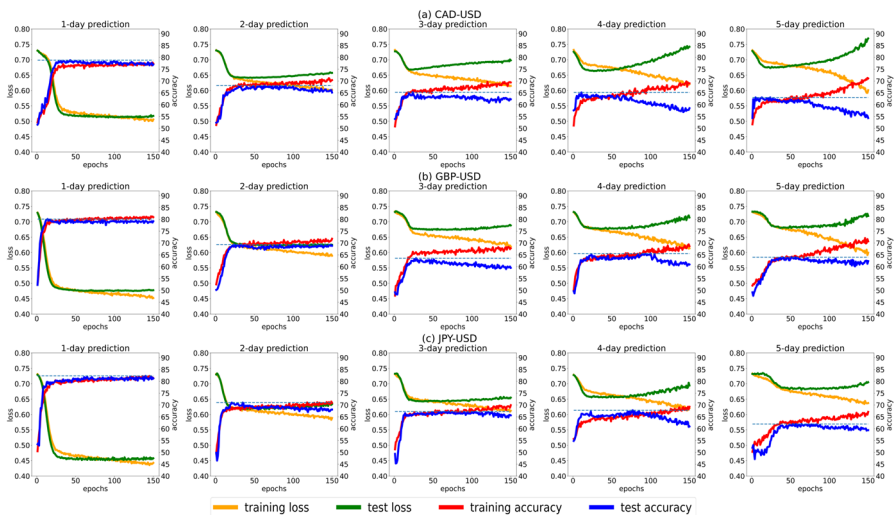
**Fig. 8** Using the lag features, our 2D-CNN-LSTM model is applied to predict the trend of three additional currency pairs: (a) CAD/USD, (b) GBP/USD, and (c) JPY/USD, arranged from top to bottom

Table 4 The highest accuracy achieved by the our method for predicting the trend of three additional Forex prices, CAD/USD, GBP/USD, and JYP/USD

Forex	1-day	2-day	3-day	4-day	5-day
CAD/USD	78.8	68.1	65.2	65.2	63.1
GBP/USD	80.1	69.4	63.6	65.6	64.0
JYP/USD	82.3	71.0	67.3	67.8	62.0

5 Conclusion

In this paper, we propose a novel deep learning framework for forecasting movements in the Forex market. Specifically, we focus on predicting the upward or downward trend of four major currency pairs, EUR/USD, CAD/USD, GBP/USD, and JPY/USD, over the next five days. Addressing this binary classification task primarily depends on developing an advanced predictive model coupled with a carefully selected set of features suited to the task. Motivated by the observation that the target trend is inherently related to the differences between data points, and considering that lag features naturally encode such information, we utilize lag features as the primary input to our model. We design a 2D-CNN-LSTM architecture, which mainly comprises a 2D convolutional neural network followed by an LSTM layer, and a final dense layer for prediction. Unlike traditional 1D-CNNs, the 2D-CNN enables the model to learn meaningful patterns along both the time and feature axes simultaneously. Additionally, an asymmetric kernel is employed to accommodate different weighting ranges across these axes, enhancing the model's ability to capture relevant dependencies. The output of the 2D-CNN is then fed into the LSTM layer to further extract temporal patterns, culminating in the dense layer that produces the final classification. For comparative purposes, we implement four classical models: MLP, LSTM, 2D-CNN, and CNN-LSTM. Experimental results show that our proposed model demonstrates superior learning capacity and achieves the highest prediction accuracy among all tested approaches.

To assess the effectiveness of the proposed lag features, we also train the models using conventional technical indicators as input features for the same prediction tasks. The experimental results clearly indicate that incorporating lag features leads to a significant improvement in prediction accuracy across all evaluated models, compared to using traditional technical indicators. Unlike standard technical indicators, which are often derived through smoothing or complex transformations of price data, the lag features we employ are raw differences computed directly from the original prices. These features preserve important information about the underlying trend and volatility, without the need for smoothing techniques. Consequently, they are more data-driven and adaptable, allowing the deep learning models to autonomously learn relevant patterns without relying on manually engineered rules. Our findings suggest that lag features, when integrated within a deep learning framework, can be both sufficient and even superior for Forex trend prediction.

The primary output of our model is a predicted change in Forex price for a specific forecast horizon (i.e., 1-day or 5-day ahead). These forecasts can be utilized

to inform trading decisions, such as initiating a long position in anticipation of an upward trend or a short position in response to a predicted downward movement. While the use of lag-1 to lag-5 features focuses on short-term Forex dynamics, our model architecture preserves longer-term trends. Specifically, combining 2D-CNN with LSTM enables the model to capture both local feature interactions and temporal dependencies. The CNN component identifies short-term patterns across multiple lag features, while the LSTM component is capable of learning sequential dependencies over time, even when input features are limited to short lags. This structure allows the model to recognize sustained movements that indicate longer-term trends, rather than overreacting to short-term fluctuations. Additionally, we employ technologies such as regularization, dropout, and early stopping to reduce the overfitting to noise and ensure more stable generalization. Together, these design choices help ensure that the model remains robust against minor volatility while still capturing the underlying trend structure in Forex time series.

While our current implementation focuses on short lag windows for efficiency and interpretability, the architecture is flexible and can be extended to include longer or multi-scale lags in future work. Moreover, we plan to apply our method to the prediction of additional financial assets, such as stock prices and various market indices. The feature set will also be expanded to include other relevant variables, such as interest rates, returns, turnover, and more. To enhance the model's ability to capture complex patterns, advanced neural network architectures, such as transformers, will be explored.

Funding This work was supported by Albion College through the Faculty Development Large Grants.

Data Availability The data used in this study is downloaded from [Yahoo Finance](#). The source code will be available on reasonable request.

Declarations

Competing interests The authors have no relevant financial or non-financial interests to disclose.

References

- Abbarbanell, J. S., & Bushee, B. J. (1997). Fundamental analysis, future earnings, and stock prices. *Journal of Accounting Research*, 35(1), 1–24.
- Ali, M., Khan, D. M., Alshanbari, H. M., & El-Bagoury, A.A.-A.H. (2023). Prediction of complex stock market data using an improved hybrid EMD-LSTM model. *Applied Sciences*, 13(3), 1429.
- Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(53).
- Anders, U., & Korn, O. (1999). Model selection in neural networks. *Neural Networks*, 12(2), 309–323.
- Ariyo, A.A., Adewumi, A.O., Ayo, C.K. (2014). Stock price prediction using the arima model. In *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, (pp. 106–112). IEEE

- Assaf, A. (2008). Nonstationarity in real exchange rates using unit root tests with a level shift at unknown time. *International Review of Economics & Finance*, 17(2), 269–278.
- Bhandari, H. N., Rimal, B., Pokhrel, N. R., Rimal, R., Dahal, K. R., & Khatri, R. K. (2022). Predicting stock market index using LSTM. *Machine Learning with Applications*, 9, 100320.
- Billah, M.M., Sultana, A., Bhuiyan, F., & Kaosar, M.G. (2024). Stock price prediction: comparison of different moving average techniques using deep learning model. *Neural Computing and Applications*, 1–11
- Brockwell, P.J., & Davis, R.A. (2002). *Introduction to Time Series and Forecasting*. Springer.
- Chandola, D., Mehta, A., Singh, S., Tikkiwal, V. A., & Agrawal, H. (2023). Forecasting directional movement of stock prices using deep learning. *Annals of Data Science*, 10(5), 1361–1378.
- Devi, B. U., Sundar, D., & Alli, P. (2013). An effective time series analysis for stock trend prediction using arima model for nifty midcap-50. *International Journal of Data Mining & Knowledge Management Process*, 3(1), 65.
- Di Persio, L., Honchar, O., et al. (2016). Artificial neural networks architectures for stock price prediction: Comparisons and applications. *International Journal of Circuits, Systems and Signal Processing*, 10, 403–413.
- Di Persio, L., Honchar, O., et al. (2017). Recurrent neural networks approach to the financial forecast of google assets. *International Journal of Mathematics and Computers in Simulation*, 11, 7–13.
- Fama, E. F. (1995). Random walks in stock market prices. *Financial Analysts Journal*, 51(1), 75–80.
- Fathali, Z., Kodia, Z., & Ben Said, L. (2022). Stock market prediction of nifty 50 index applying machine learning techniques. *Applied Artificial Intelligence*, 36(1), 2111134.
- Fischer, T., Sterling, M., & Lessmann, S. (2024). Fx-spot predictions with state-of-the-art transformer and time embeddings. *Expert Systems with Applications*, 249, 123538.
- Fogel, D. B. (1991). An information criterion for optimal neural network selection. *IEEE Transactions on Neural Networks*, 2(5), 490–497.
- Gudelek, M.U., Boluk, S.A., & Ozbayoglu, A.M. (2017). A deep learning based stock trading model with 2-d cnn trend detection. In *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*, (pp. 1–8). IEEE
- Han, Y., Liu, Y., Zhou, G., & Zhu, Y. (2021). *Technical analysis in the stock market: A review*. <https://doi.org/10.2139/ssrn.3850494>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Hoseinzade, E., & Haratizadeh, S. (2019). Cnnpred: Cnn-based stock market prediction using a diverse set of variables. *Expert Systems with Applications*, 129, 273–285.
- Hu, Z., Zhao, Y., & Khushi, M. (2021). A survey of forex and stock price prediction using deep learning. *Applied System Innovation*, 4(1), 9.
- Igboanusi, M.O. (2024). *Artificial intelligence application to euro-usd forex value prediction using lstm tensor flow*
- Jarrah, M., & Derbali, M. (2023). Predicting saudi stock market index by using multivariate time series based on deep learning. *Applied Sciences*, 13(14), 8356.
- Jiang, W. (2021). Applications of deep learning in stock market prediction: recent progress. *Expert Systems with Applications*, 184, 115537.
- Kaur, G., Dhar, J., & Guha, R. K. (2016). Minimal variability owa operator combining anfis and fuzzy c-means for forecasting bse index. *Mathematics and Computers in Simulation*, 122, 69–80.
- Kelotra, A., & Pandey, P. (2020). Stock market prediction using optimized deep-convlstm model. *Big Data*, 8(1), 5–24.
- Khan, W., Ghazanfar, M.A., Azam, M.A., Karami, A., Alyoubi, K.H., & Alfakeeh, A.S. (2022). Stock market prediction using machine learning classifiers and social media, news. *Journal of Ambient Intelligence and Humanized Computing*, 1–24
- Khatun, A., Nisha, M. N., Chatterjee, S., & Sridhar, V. (2024). A novel insight on input variable and time lag selection in daily streamflow forecasting using deep learning models. *Environmental Modelling & Software*, 179, 106126.
- Kong, X., Chen, Z., Liu, W., Ning, K., Zhang, L., Marier, S.M., Liu, Y., Chen, Y., & Xia, F. (2025). Deep learning for time series forecasting: a survey. *International Journal of Machine Learning and Cybernetics*
- LeCun, Y., Bengio, Y., et al. (1995). Convolutional networks for images, speech, and time series. *The Handbook of Brain Theory and Neural Networks*, 3361(10), 1995.

- Livieris, I. E., Stavroyiannis, S., Pintelas, E., & Pintelas, P. (2020). A novel validation framework to enhance deep learning models in time-series forecasting. *Neural Computing and Applications*, 32(23), 17149–17167.
- Lu, W., Li, J., Li, Y., Sun, A., & Wang, J. (2020). A cnn-lstm-based model to forecast stock prices. *Complexity*, 2020, 1–10.
- Lu, W., Li, J., Wang, J., & Qin, L. (2021). A cnn-bilstm-am method for stock price prediction. *Neural Computing and Applications*, 33, 4741–4753.
- Malkiel, B.G. (1999). *A Random Walk down Wall Street: Including a Life-cycle Guide to Personal Investing*. WW Norton & Company
- Miller, J.A., Aldosari, M., Saeed, F., Barna, N.H., Rana, S., Arpinar, I.B., Liu, N. (2024). *A survey of deep learning and foundation models for time series forecasting*. <https://doi.org/10.48550/arXiv.2401.13912>
- Moreno-Pino, F., Olmos, P. M., & Artés-Rodríguez, A. (2023). Deep autoregressive models with spectral attention. *Pattern Recognition*, 133, 109014.
- Nabipour, M., Nayyeri, P., Jabani, H., Shahab, S., & Mosavi, A. (2020). Predicting stock market trends using machine learning and deep learning algorithms via continuous and binary data; a comparative analysis. *IEEE Access*, 8, 150199–150212.
- Navaei, M., & Pahlevanzadeh, M. (2024). Forecasting next-time-step forex market stock prices using neural networks. *Advances in Machine Learning & Artificial Intelligence*, 5(2), 1–10.
- Nikou, M., Mansourfar, G., & Bagherzadeh, J. (2019). Stock price prediction using deep learning algorithm and its comparison with machine learning algorithms. *Intelligent Systems in Accounting, Finance and Management*, 26(4), 164–174.
- Rodrigues, P. C., Awe, O. O., Pimentel, J. S., & Mahmoudvand, R. (2020). Modelling the behaviour of currency exchange rates with singular spectrum analysis and artificial neural networks. *Stats, MDPI*, 3(2), 137–157.
- Rosillo, R., Giner, J., & Fuente, D. (2014). The effectiveness of the combined use of vix and support vector machines on the prediction of s & p 500. *Neural Computing and Applications*, 25, 321–332.
- Sedighi, M., Jahangirnia, H., Gharakhani, M., & Farahani Fard, S. (2019). A novel hybrid model for stock price forecasting based on metaheuristics and support vector machine. *Data*, 4(2), 75.
- Sen, J., Mehtab, S., & Nath, G. (2023). *Stock price prediction using deep learning models*. Authorea Preprints
- Taylor, M. P., & Allen, H. (1992). The use of technical analysis in the foreign exchange market. *Journal of International Money and Finance*, 11(3), 304–314.
- Vantuch, T., Zelinka, I. (2015). Evolutionary based arima models for stock price forecasting. In *ISCS 2014: Interdisciplinary Symposium on Complex Systems*, (pp. 239–247). Springer
- Wafi, A. S., Hassan, H., & Mabrouk, A. (2015). Fundamental analysis models in financial markets – review study. *Procedia Economics and Finance*, 30, 939–947.
- Wang, C., Chen, Y., Zhang, S., & Zhang, Q. (2022). Stock market index prediction using deep transformer model. *Expert Systems with Applications*, 208, 118128.
- Wang, J., Wang, X., Li, J., & Wang, H. (2021). A prediction model of cnn-tlstm for usd/cny exchange rate prediction. *IEEE Access*, 9, 73346–73354.
- White, H. (1988). Economic prediction using neural networks: The case of ibm daily stock returns. In: *ICNN*, (vol. 2, pp. 451–458).
- Zaheer, S., Anjum, N., Hussain, S., Algarni, A. D., Iqbal, J., Bourouis, S., & Ullah, S. S. (2023). A multi parameter forecasting for stock time series data using LSTM and deep learning model. *Mathematics*, 11(3), 590.
- Zhao, Y., & Yang, G. (2023). Deep learning-based integrated framework for stock price movement prediction. *Applied Soft Computing*, 133, 109921.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Bilguun Narmandakh¹ · Yuming Zhang¹  · Zhen Li² · Paul Anderson¹

✉ Yuming Zhang
yzhang@albion.edu

Bilguun Narmandakh
bn12@albion.edu

Zhen Li
zli@albion.edu

Paul Anderson
panderson@albion.edu

¹ Department of Mathematics and Computer Science, Albion College, 611 E. Porter St.,
Albion 49224, Michigan, USA

² Department of Economics and Management, Albion College, 611 E. Porter St.,
Albion 49224, Michigan, USA